

Rezolvare Completă și Detaliată

Examenul Național pentru Definitivare în Învățământ

Anul 2018 - Informatică și Tehnologia Informației

Cuprins

I. Rezolvare Definitivat Varianta 2 (18 iulie 2018)	2
SUBIECTUL I (60 de puncte)	2
1. Structura de date: Arbori Binari de Căutare (ABC)	2
2. Procesoare de text: Formatare text	2
3. Programare: Reprezentare sumă termeni recurență	3
4. Baze de date: Florărie	5

I. Rezolvare Definitivat Varianta 2 (18 iulie 2018)

[Subiect PDF](file:

SUBIECTUL I (60 de puncte)

1. Structura de date: Arbori Binari de Căutare (ABC)

- **Definire:** Un arbore binar este arbore de căutare dacă cheia fiecărui nod este strict mai mare decât cheile tuturor nodurilor din subarboarele stâng și strict mai mică sau egală cu cheile tuturor nodurilor din subarboarele drept.
- **Operații:**
 - **Căutare/Inserare:** Pornind de la rădăcină, comparăm cheia. Dacă este mai mică mergem la stânga, dacă este mai mare mergem la dreapta.
 - **Ștergere:** Cazul în care nodul este frunză (se elimină direct), are un singur fiu (se înlocuiește cu fiul său) sau are doi fii (se înlocuiește cu cel mai mare element din subarboarele stâng sau cel mai mic din subarboarele drept).
 - **Parcurgeri:** Inordine (sortat crescător), Preordine, Postordine.
- **Problemă (Gestiune dicționar):**
 - **C++:**

```
#include <iostream>
using namespace std;
struct Nod {
    int cheie;
    Nod *st, *dr;
};
void inserare(Nod* &rad, int x) {
    if (rad == nullptr) rad = new Nod{x, nullptr, nullptr};
    else if (x < rad->cheie) inserare(rad->st, x);
    else inserare(rad->dr, x);
}
void inordine(Nod* rad) {
    if (rad != nullptr) {
        inordine(rad->st);
        cout << rad->cheie << " ";
        inordine(rad->dr);
    }
}
int main() {
    Nod* rad = nullptr;
    inserare(rad, 10); inserare(rad, 5); inserare(rad, 15);
    inordine(rad);
    return 0;
}
```

2. Procesoare de text: Formatare text

- **Elemente inserare:** Celulă tabel, casetă de text (Text Box), antet/subsol (Header/Footer).
- **Formatare paragraf:** Aliniere, indentare, spațiere rânduri, spațiere paragrafe, chenare.
- **Formatare caracter:** Font, dimensiune, culoare, stil (bold/italic/underline), spațiere caractere.
- **Metode aplicare:** Formatare directă, stiluri (Styles).

3. Programare: Reprezentare sumă termeni recurență

Soluție C++:

```
#include <iostream>
#include <fstream>
#include <vector>
using namespace std;

long long termen(long long n) {
    long long prev2 = 1, prev1 = 2;
    if (n < 1) return 0;
    if (n == 1) return 1;
    if (n == 2) return 2;
    while (true) {
        long long curr = 1 + 3 * prev1 - 2 * prev2;
        if (curr > n) return prev1;
        prev2 = prev1;
        prev1 = curr;
    }
}

int main() {
    long long n;
    if (!(cin >> n)) return 0;

    vector<long long> F;
    F.push_back(1);
    F.push_back(2);
    while (true) {
        long long next_val = 1 + 3 * F.back() - 2 * F[F.size() - 2];
        if (next_val > 1000000000LL) break;
        F.push_back(next_val);
    }

    vector<long long> rez;
    long long sum = n;
    for (int i = F.size() - 1; i >= 0; --i) {
        if (sum >= F[i]) {
            rez.push_back(F[i]);
            sum -= F[i];
        }
        if (sum >= F[i]) {
            rez.push_back(F[i]);
            sum -= F[i];
        }
    }

    ofstream fout("def.out");
    for (int i = 0; i < rez.size(); ++i) {
        fout << rez[i] << (i == rez.size() - 1 ? "" : " ");
    }
    fout.close();
}
```

```
    return 0;
}
```

Soluție Pascal:

```
program SumaTermeni;
var
    n, sum_val: int64;
    F: array[1..40] of int64;
    rez: array[1..80] of int64;
    f_count, rez_count, i, j: integer;
    fout: text;

function termen(n_val: int64): int64;
var
    prev2, prev1, curr: int64;
begin
    if n_val < 1 then termen := 0
    else if n_val = 1 then termen := 1
    else if n_val = 2 then termen := 2
    else
        begin
            prev2 := 1; prev1 := 2;
            while true do
                begin
                    curr := 1 + 3 * prev1 - 2 * prev2;
                    if curr > n_val then
                        begin
                            termen := prev1;
                            exit;
                        end;
                    prev2 := prev1; prev1 := curr;
                end;
            end;
        end;
end;

begin
    readln(n);
    F[1] := 1; F[2] := 2;
    f_count := 2;
    while true do
        begin
            F[f_count + 1] := 1 + 3 * F[f_count] - 2 * F[f_count - 1];
            if F[f_count + 1] > 1000000000 then break;
            f_count := f_count + 1;
        end;

    sum_val := n;
    rez_count := 0;
    for i := f_count downto 1 do
        begin
            if sum_val >= F[i] then
                begin
                    rez_count := rez_count + 1;
                    rez[rez_count] := F[i];
                    sum_val := sum_val - F[i];
                end;
        end;
```

```

if sum_val >= F[i] then
begin
    rez_count := rez_count + 1;
    rez[rez_count] := F[i];
    sum_val := sum_val - F[i];
end;
end;

assign(fout, 'def.out');
rewrite(fout);
for i := 1 to rez_count do
begin
    write(fout, rez[i]);
    if i < rez_count then write(fout, ' ');
end;
close(fout);
end.

```

4. Baze de date: Florărie

• Entități:

- ▶ CLIENT: id_client (PK), nume, prenume, adresa, telefon.
- ▶ TIP_FLOARE: id_floare (PK), denumire stiintifica, pret_unitar, anotimp_specific.
- ▶ COMANDA: id_client (FK), id_floare (FK), data_comanda, cantitate, PK este (id_client, id_floare, data_comanda).

• SQL:

```

SELECT denumire_stiintifica, pret_unitar
FROM TIP_FLOARE
WHERE anotimp_specific = 'vara';

```